

AI Operating System Workshop

Day 2: Knowledge Graph & Advanced RAG

Knowledge Graph และ RAG ขั้นสูง

X-Company

2026

ส่วนที่ 1: Knowledge Graph พื้นฐาน

Knowledge Graph (KG) คือการแสดงข้อมูลเชิงโครงสร้างของ entities ในโลกจริงและความสัมพันธ์ระหว่างกัน จัดระเบียบข้อมูลเป็นเครือข่าย nodes และ edges ช่วยให้ทำ multi-hop reasoning ที่ Vector DB ไม่สามารถทำได้

- Nodes: Entities — บุคคล, องค์กร, สินค้า, แนวคิด
- Edges: ความสัมพันธ์ — 'ทำงานที่', 'เป็นส่วนหนึ่งของ', 'เกี่ยวข้องกับ'
- Properties: คุณสมบัติของ nodes/edges — ชื่อ, วันที่, คะแนนความเชื่อมั่น

KG vs Relational DB vs Vector DB

คุณลักษณะ	Knowledge Graph	Relational DB	Vector DB
โมเดลข้อมูล	กราฟ (nodes/edges)	ตาราง/แถว	Vector หลายมิติ
ประเภท Query	Traversal, pattern match	SQL joins	Similarity search
Multi-hop	Native, เร็ว	JOIN ซับซ้อน	ไม่รองรับ
Schema	ยืดหยุ่น (schema-less)	Schema แข็งกร้าว	ไม่มี schema
เหมาะกับ	ความสัมพันธ์, reasoning	Transaction, CRUD	Semantic search
ตัวอย่าง	Neo4j, Amazon Neptune	PostgreSQL, MySQL	Qdrant, Pinecone

ส่วนที่ 2: การดึง Entity

NER ด้วย spaCy

```
import spacy

nlp = spacy.load('en_core_web_sm')
doc = nlp('Apple CEO Tim Cook announced new products at WWDC in San Francisco.')

for ent in doc.ents:
    print(f'{ent.text:20} | {ent.label_:10} | {spacy.explain(ent.label_)}')

# ผลลัพธ์:
# Apple | ORG | บริษัท, หน่วยงาน
# Tim Cook | PERSON | บุคคล
# WWDC | EVENT | งานอีเวนต์
# San Francisco | GPE | ประเทศ, เมือง
```

การดึง Entity ด้วย LLM

ใช้ LLM กับ prompt แบบ structured output เพื่อการดึง entity ที่ยืดหยุ่นและเข้าใจ context:

```
import ollama, json

prompt = '''

ดึง entities จากข้อความ ส่งกลับในรูปแบบ JSON:
{"entities": [{"name": str, "type": str, "description": str}],
"relations": [{"source": str, "relation": str, "target": str}]}

ข้อความ: {text}
'''

response = ollama.chat(
    model='llama3.2',
    messages=[{'role': 'user', 'content': prompt.format(text=doc_text)}],
    format='json'
)
data = json.loads(response['message']['content'])
```

ประเภท Entity อ้างอิง

ประเภท Entity	ตัวอย่าง	วิธีการดึง
PERSON	สมชาย, Tim Cook, Elon Musk	NER / LLM
ORG	Apple, X-Company, OpenAI	NER / LLM
LOCATION	กรุงเทพ, Silicon Valley	NER / LLM
PRODUCT	iPhone, ChatGPT, Qdrant	LLM (เข้าใจ context)
CONCEPT	Machine Learning, RAG	LLM (domain-specific)
EVENT	WWDC, CES, สัมมนา	NER / LLM

ส่วนที่ 3: การสร้าง Knowledge Graph

กระบวนการสร้าง Knowledge Graph ประกอบด้วยหลายขั้นตอน:

- ขั้นที่ 1: รับเอกสารและแบ่ง chunk
- ขั้นที่ 2: ดึง entities จากแต่ละ chunk
- ขั้นที่ 3: ดึงความสัมพันธ์ระหว่าง entities
- ขั้นที่ 4: Entity resolution (ลบซ้ำ)
- ขั้นที่ 5: เก็บ graph ใน Neo4j

```
# ขั้นที่ 1-2: ดึง entities จากเอกสาร
entities, relations = [], []
for chunk in chunks:
    result = extract_entities_llm(chunk)
    entities.extend(result['entities'])
    relations.extend(result['relations'])
```

```
# ขั้นที่ 5: เก็บใน Neo4j
from neo4j import GraphDatabase
driver = GraphDatabase.driver('bolt://localhost:7687',
                             auth=('neo4j', 'password'))
with driver.session() as session:
    for e in entities:
        session.run('MERGE (n:{type} {{name: $name}})',
                    type=e['type'], name=e['name'])
    for r in relations:
        session.run(
            'MATCH (a {{name: $src}}), (b {{name: $tgt}})'
            'MERGE (a)-[:{rel}]->(b)',
            src=r['source'], tgt=r['target'],
            rel=r['relation'].upper())
```

ส่วนที่ 4: Neo4j และ Cypher

ติดตั้ง Neo4j

```
# เริ่มต้น Neo4j ด้วย Docker
docker run -d --name neo4j \
  -p 7474:7474 -p 7687:7687 \
  -e NEO4J_AUTH=neo4j/password \
  -v $(pwd)/neo4j_data:/data \
  neo4j:5.13

# เข้า Neo4j Browser: http://localhost:7474
```

พื้นฐาน Cypher

```
// สร้าง node
CREATE (p:Person {name: 'สมชาย', role: 'วิศวกร'})

// MATCH และ RETURN
MATCH (p:Person) WHERE p.name = 'สมชาย' RETURN p

// สร้าง relationship
MATCH (a:Person {name: 'สมชาย'}), (b:Company {name: 'X-Company'})
CREATE (a)-[:WORKS_AT {since: 2020}]->(b)

// ค้นหา 2 ชั้น
MATCH (p:Person)-[:WORKS_AT]->(c:Company)-[:USES]->(t:Technology)
RETURN p.name, c.name, t.name
```

10 ตัวอย่าง Cypher Queries

#	กรณีใช้งาน	Query
---	------------	-------

1	หาบุคคลทั้งหมด	MATCH (p:Person) RETURN p.name
2	หาพนักงานขององค์กร	MATCH (p)-[:WORKS_AT]->(c {name:'X-Company'}) RETURN p
3	นับ nodes ตามประเภท	MATCH (n) RETURN labels(n), count(n)
4	ค้นหาเชื่อมต่อ 2 ชั้น	MATCH (a)-[*2]-(b) WHERE a.name='Alice' RETURN b
5	เพื่อนร่วมงาน	MATCH (a)-[:WORKS_AT]->(c)-[:WORKS_AT]-(b) RETURN a,b
6	ลบ node	MATCH (n:Person {name:'Alice'}) DETACH DELETE n
7	อัปเดตคุณสมบัติ	MATCH (n {name:'Alice'}) SET n.role='Manager' RETURN n
8	หา isolated nodes	MATCH (n) WHERE NOT (n)--() RETURN n
9	รวม relations	MATCH ()-[r]->() RETURN type(r), count(r) ORDER BY count(r) DESC
10	เส้นทางสั้นสุด	MATCH p=shortestPath((a {name:'Alice'})-[*]-(b {name:'Bob'})) RETURN p

ส่วนที่ 5: GraphRAG

GraphRAG รวม vector similarity search กับ graph traversal เพื่อตอบคำถามซับซ้อนที่ต้องการ reasoning ข้ามเอกสารหลายไฟล์

สถาปัตยกรรม:

- Query → Embed → Vector search (ค้นหา nodes ที่ใกล้เคียง)
- Nodes ที่ดึงมา → Graph traversal (ขยาย context ผ่านความสัมพันธ์)
- Context ที่ขยายแล้ว → LLM → คำตอบ

การกำหนดค่า GraphRAG-rs

```
# graphrag-rs config.toml
[storage]
  [storage.neo4j]
    uri = 'bolt://localhost:7687'
    username = 'neo4j'
    password = 'password'

  [storage.qdrant]
    url = 'http://localhost:6333'
    collection = 'workshop_docs'

[embedding]
  model = 'bge-m3'
  dimensions = 1024

[retrieval]
```

```
top_k = 5
graph_hops = 2
community_level = 2
```

ส่วนที่ 6: Advanced Retrieval

Multi-Hop Reasoning

การ reasoning บนกราฟช่วยตอบคำถามที่ต้องเชื่อมข้อมูลจากหลายเอกสาร:

ตัวอย่าง: "สมาชิกทีมคนไหนทำงานในโปรเจกต์ที่ใช้ Python และส่งมอบใน Q4?"

- ขั้น 1: หาโปรเจกต์ทั้งหมดที่ส่งใน Q4 (graph query)
- ขั้น 2: กรองเฉพาะที่ใช้ Python (graph property)
- ขั้น 3: หาสมาชิกทีมที่เชื่อมกับโปรเจกต์เหล่านั้น

Community Detection

ระบุกลุ่ม entities ที่เกี่ยวข้องกันในกราฟ ช่วยสรุปข้อมูลระดับหัวข้อ

```
import networkx as nx
from networkx.algorithms.community import greedy_modularity_communities

G = nx.Graph()
for rel in relations:
    G.add_edge(rel['source'], rel['target'])

communities = list(greedy_modularity_communities(G))

print(f'พบ {len(communities)} กลุ่ม')
```

Graph-Enhanced Context

```
def graph_enhanced_retrieval(query: str) -> str:
    # 1. Vector search
    vector_chunks = vector_search(query, top_k=3)

    # 2. ดึง entities จาก query
    entities = extract_entities_llm(query)

    # 3. ขยาย context จากกราฟ
    with driver.session() as session:
        for entity in entities:
            result = session.run(
                'MATCH (n {name:$name})-[r*1..2]-(m) '
                'RETURN m.name, type(r[-1]) LIMIT 10',
                name=entity['name']
            )
    return combined_context
```

ส่วนที่ 7: แบบฝึกหัดและเอกสารอ้างอิง

แบบฝึกหัดที่ 1: สร้าง **Company Knowledge Graph**

งาน: ดึง entities จากเอกสาร 10 ไฟล์ และสร้าง Neo4j graph

- รวม: ประเภท node — Person, Organization, Project, Technology
- Hint: ใช้ LLM extraction พร้อม JSON output format

แบบฝึกหัดที่ 2: **Cypher Query Challenge**

งาน: เขียน Cypher queries เพื่อตอบ: พนักงาน 3 คนที่เชื่อมต่อกันมากที่สุดคือใคร? เทคโนโลยีใดปรากฏในโปรเจกต์มากที่สุด?

- Hint: ใช้ฟังก์ชัน degree() หรือนับความสัมพันธ์ใน MATCH

แบบฝึกหัดที่ 3: **GraphRAG Pipeline**

งาน: สร้าง GraphRAG pipeline ที่ตอบคำถาม multi-hop โดยใช้ knowledge graph

- Hint: รวม Qdrant search กับ Neo4j 2-hop expansion

เอกสารอ้างอิง

- Neo4j Documentation: <https://neo4j.com/docs>
- GraphRAG paper: <https://arxiv.org/abs/2404.16130>
- spaCy NER: <https://spacy.io/usage/linguistic-features>
- NetworkX: <https://networkx.org/documentation>